



Master Advanced Machine Learning and Multimedia Intelligence

Developing a University Timetabling System

A Step-by-Step Guide Using Spring Boot, Java, and MySQL, Based on Integer Programming Formulation

Mohammed El Idrissi Laoukili

Supervisor:

Prof. LOQMAN CHAKIR

Academic Year: 2024/2025

Table of Contents

Summary	3
General Introduction	4
I. Application modeling and database design	5
1. <i>General features of the model</i>	5
2. <i>Special features of the model</i>	5
3. <i>Core Entities and Relationships</i>	6
4. <i>Database diagram</i>	7
5. <i>Extending the Database to a Pro Version</i>	8
6. <i>Database pro version diagram:</i>	9
7. <i>SQL Queries for Data Retrieval and Management</i>	10
8. <i>Data variables</i>	11
II. Development Tools & project structure	13
1. <i>Tools:</i>	13
2. <i>Project structure:</i>	14
III. Testing and Debugging Approach	15
IV. Applying the Proposed Model	16
1. Timetable Diagram	16
2. <i>Constraints for the IP model</i>	19
2.1. Uniqueness constraints	19
2.2. Completeness constraints	20
2.3. Consecutiveness constraints	24
2.4. Repetitiveness constraints	24
2.5. Pre-assignment constraints	25
2.6. Objective function for the IP model	25
2.7. Running the program using on simple data	25
3. Proposed Real-World Datasets for Testing	27
3.1. Lower-Grade Dataset:	27
3.2. Higher-Grade Dataset:	29
V. Running timetabling application with spring boot	30
Conclusion	32

Summary

This report represents the outcome of our work over the past month. The objective of this project is to design and implement a web application for generating university timetables. The methodology adopted was inspired by the research presented in the article, *"An Integer Programming Formulation for a Case Study in University Timetabling"* by S. Daskalaki, T. Birbas, and E. Housos. The article provided insights into modeling university timetabling as a 0-1 integer programming problem, addressing operational constraints and optimization to ensure feasible and quality schedules.

Before entering the development phase, we began by analyzing requirements and defining specifications, guided by the concepts in the article. Following this, we conducted a detailed design based on the Unified Modeling Language (UML). This study allowed us to establish the technical choices adopted in the development process and to create a structured work plan, enabling us to achieve good results while adhering to quality standards.

This report also demonstrates how the proposed system was incrementally developed, starting with simple terminal-based data demonstrations, progressing to database integration, and finally presenting a full-stack implementation using Spring Boot, Java, and MySQL.

General Introduction

Thanks to the rapid advancements in computer science and the emergence of cutting-edge technologies, numerous industries are increasingly adopting software solutions to enhance efficiency, automate processes, and meet modern demands. The education sector is one such area where scheduling, a traditionally manual and error-prone task, can greatly benefit from technological innovation.

Application Cures What ?

Timetabling in universities is a challenging task. Ensuring that classes, teachers, and students are scheduled efficiently while adhering to numerous constraints is a time-consuming and complex process. Administrative staff often replicate old timetables with minor adjustments, which is not always sufficient to handle dynamic changes like new courses, staff availability, or classroom allocation.

This is where automated timetabling systems come into play. By leveraging optimization algorithms, such systems can generate conflict-free schedules that meet both operational rules and user preferences. This reduces the administrative burden while improving the quality and satisfaction of the timetable.

Idea of the Application

Our project focuses on solving the university timetabling problem by designing a platform that automates the generation of timetables. The primary goal is to ensure efficient scheduling of courses, teachers, and classrooms while adhering to operational and quality constraints. The system is designed to provide administrators with a streamlined solution that eliminates manual errors and saves time.

Purpose of the Project

The objective of this project is to develop an efficient and user-friendly system for generating university timetables. By leveraging optimization techniques such as Integer Programming (IP), the application ensures that the generated timetables are both feasible and satisfactory for the institution.

How It Works

Once the data, such as courses, classrooms, teachers, and availability, is inserted into the database using CRUD operations, the system can start the process of timetable generation. A single click initiates the process, where only the rows marked with an active state (e.g., state = 1) are queried and passed to the Integer Programming (IP) module. The IP program then generates a conflict-free timetable that meets all specified constraints.

I. Application modeling and database design

✍ Firstly, we will discuss the approach we followed in undertaking this project, which involves the application of the Unified Modeling Language (UML). UML provides several advantages by offering a set of design diagrams that help visualize and structure the project effectively. These diagrams facilitate clear communication and understanding of the system's architecture, roles, and processes, which are crucial for ensuring the proper implementation of the system.

Building on the provided article, the database for the university timetabling system is designed to align with the described Integer Programming (IP) model. This alignment ensures that the database incorporates the core parameters and constraints of the timetabling problem while also supporting flexibility and scalability to accommodate different institutions' needs.

To structure the database effectively, we leverage the key entities identified in the IP model: **days of the week, time periods, student groups, teachers, courses, and classrooms**. UML class diagrams are employed to represent these entities and their relationships. Each entity is broken down into its relevant attributes and associations, which are then translated into database tables and relationships.

Additionally, the database is designed to support special features like **grouped student sets, teacher availability, and multi-session courses**. UML activity and sequence diagrams help visualize these constraints and processes, ensuring that the database schema is optimized for handling complex relationships such as student-teacher-course assignments, room allocation, and time period management.

By combining UML's modeling tools with the constraints outlined in the IP model, the database structure is robust, adaptable, and capable of supporting a university timetabling system that meets diverse institutional requirements while maintaining efficiency in constraint checking and data management.

1. General features of the model

I = The day of the week

J = The time period of a day

K = The group of students

L = The professor, lecturer or other teaching staff

M = courses

N = classrooms

2. Special features of the model

$K^1 = \{k \in K : k = \text{group of students of lower-grade years, usually students from the first two or three years, following general education within a department}\}.$

$K^2 = \{k \in K : k = \text{group of students of higher-grade years (registered to a specific division)}\}.$

It is apparent that K^1 and K^2 are disjoint sets, while $K = K^1 \cup K^2$.

$K_l = \{k \in K : k = \text{group of students (from either } K^1 \text{ or } K^2) \text{ for which teacher } l \text{ offers some course}\}.$

$L_i = \{l \in L : l = \text{teacher available on day } i\}.$

$L_k = \{l \in L : l = \text{teacher teaching at least one course for the group of students } k, k \in K\}.$

$L_m = \{l \in L : l = \text{teacher teaching course } m\}.$

$L_{km} = L_k \cap L_m \text{ and } L_{ki} = L_k \cap L_i.$

$M_k = \{m \in M : m = \text{course designed for the } k \text{ th group of students or offered by the } k \text{ th division for its students, } k \in K\}$.

$M_l = \{m \in M : m = \text{course taught by teacher } l\}$.

$M_n = \{m \in M : m = \text{course designed for a group of students that fits in classroom } n \in N\}$.

$M_{kl} = M_k \cap M_l, M_{kn} = M_k \cap M_n$, and $M_{kln} = M_k \cap M_l \cap M_n$.

$M_k^{com} = \{(k_a, l, m) \in K^2 \times L \times M : m = \text{course taught by teacher } l \text{ for the student group } k_a \in K^2, \text{ offered additionally as elective course for the student group } k \in K^2\}$. $M_{lab} = \{m \in M : m = \text{course requiring lab work}\}$.

$N_{mk} = \{n \in N : n = \text{classroom that fits the group of students } k \in K \text{ for the course } m \in M\}$.

$I_n = \{i \in I : i = \text{day on which classroom } n \text{ is available for use}\}$.

$I_l = \{i \in I : i = \text{day on which teacher } l \in L \text{ is available for teaching assignments}\}$.

$I_{ln} = I_l \cap I_n$.

$J_{iln} = \{j \in J : j = \text{time period of day } i \text{ on which teacher } l \in L \text{ and classroom } n \in N \text{ are available for assignments}\}$.

$JL_{iln} = \{(j_a, j_b) \in J_{iln} \times J_{iln} : (j_a, j_b) = \text{time interval from period } j_a \text{ to } j_b \text{ of day } i \text{ on which teacher } l \in L \text{ and classroom } n \in N \text{ are available for assignments}\}$.

$FJL_{iln} = \{j_a \in J_{iln} : j_a = \text{the starting period of an interval of day } i \text{ on which teacher } l \in L \text{ and classroom } n \in N \text{ are available for assignments}\}$.

$P_m = \{p_v \in \{1, 2, \dots, p_{max_m}\}, \text{ where } p_{max_m} \text{ is the number of repetitions required for a multi-period session of course } m \in M\}$.

$H_m = \{h_v \in N^+ : h_v = \text{length (number of periods) of a multi-period session requested for course } m \in M\}$.

$PRA = \{(i, j, k, l, m, n) \in I \times J \times K \times L \times M \times N / (i, j, k, l, m, n) = \text{an a priori assignment in the timetable, that is, course } m \text{ taught by teacher } l \text{ for the group of students } k \text{ is pre-assigned to period } j \text{ of day } i\}$.

3. Core Entities and Relationships

	Features	Corresponding Database Tables
1	I	days_of_the_week
2	J	time_periods_of_day
3	$K = K^1 \cup K^2$	groups
4	L	professors
5	M	courses
6	N	classrooms
7	K^1 & K^2	grades
8	K_l & L_k	group_prof
9	L_i & I_l	prof_available_on
10	L_m & M_l	prof_course
11	$L_{km} = L_k \cap L_m$ & $M_{kl} = M_k \cap M_l$	prof_group_course
12	$L_{ki} = L_k \cap L_i$	prof_group_day
13	M_k & K_m	course_group
14	M_n & N_m	course_classroom
15	$M_{kn} = M_k \cap M_n$ & N_{mk}	course_group_classroom
16	$M_{kln} = M_k \cap M_l \cap M_n$	course_group_prof_classroom
17	I_n & N_i	classroom_available_on
18	$I_{ln} = I_l \cap I_n$	prof_classroom_availability
19	J_{iln}	period_prof_classroom_availability
20	PRA	timetable

For the rest of features we will query them.

5. Extending the Database to a Pro Version

While the current database is sufficient for generating a timetable, it is not comprehensive enough to meet the full needs of a school or university. To address this, I have decided to extend the database to a "Pro Version" that incorporates additional features and functionalities required for a complete educational management system.

Limitations of the Current Database:

1. *Lack of User Roles and Permissions:*
The current system does not differentiate between users (e.g., administrators, professors, students), limiting its ability to control access and manage responsibilities effectively.
2. *Incomplete Group-Student Relationships:*
Student groups are not explicitly linked to individual students, making it difficult to track attendance, performance, or other group-specific activities.
3. *Missing Features for Comprehensive Management:*
The database lacks essential features such as grading systems which is critical for a fully functional educational institution.

Features of the Pro Version:

To address these limitations, the Pro Version of the database will include the following enhancements:

1. *User Roles and Permissions:*
 - Introduce role-based access control (RBAC) to define permissions for administrators, professors, and students.
 - Ensure that users can only access and modify data relevant to their roles.
2. *Student-Group Relationships:*
 - Link student groups to individual students, enabling better tracking of group activities and performance.
 - Allow students to belong to multiple groups (e.g., classes, clubs, or project teams).
3. *Course Enrollment and Management:*
 - Provide professors with tools to manage course content, assignments, and grades.
4. *Scalability and Flexibility:*
 - Design the database to handle larger datasets and more complex relationships, ensuring scalability for growing institutions.
 - Provide flexibility to customize the system based on the specific needs of different schools or universities.

Benefits of the Pro Version:

- *Comprehensive Management:* The enhanced database will serve as a complete solution for managing all aspects of an educational institution, from scheduling to student performance tracking.
- *Improved User Experience:* Role-based access and intuitive interfaces will make the system easier to use for administrators, professors, and students.

6. Database pro version diagram:

For better view of the schema: <https://dbdiagram.io/d/school-67790d0a5406798ef74210de>

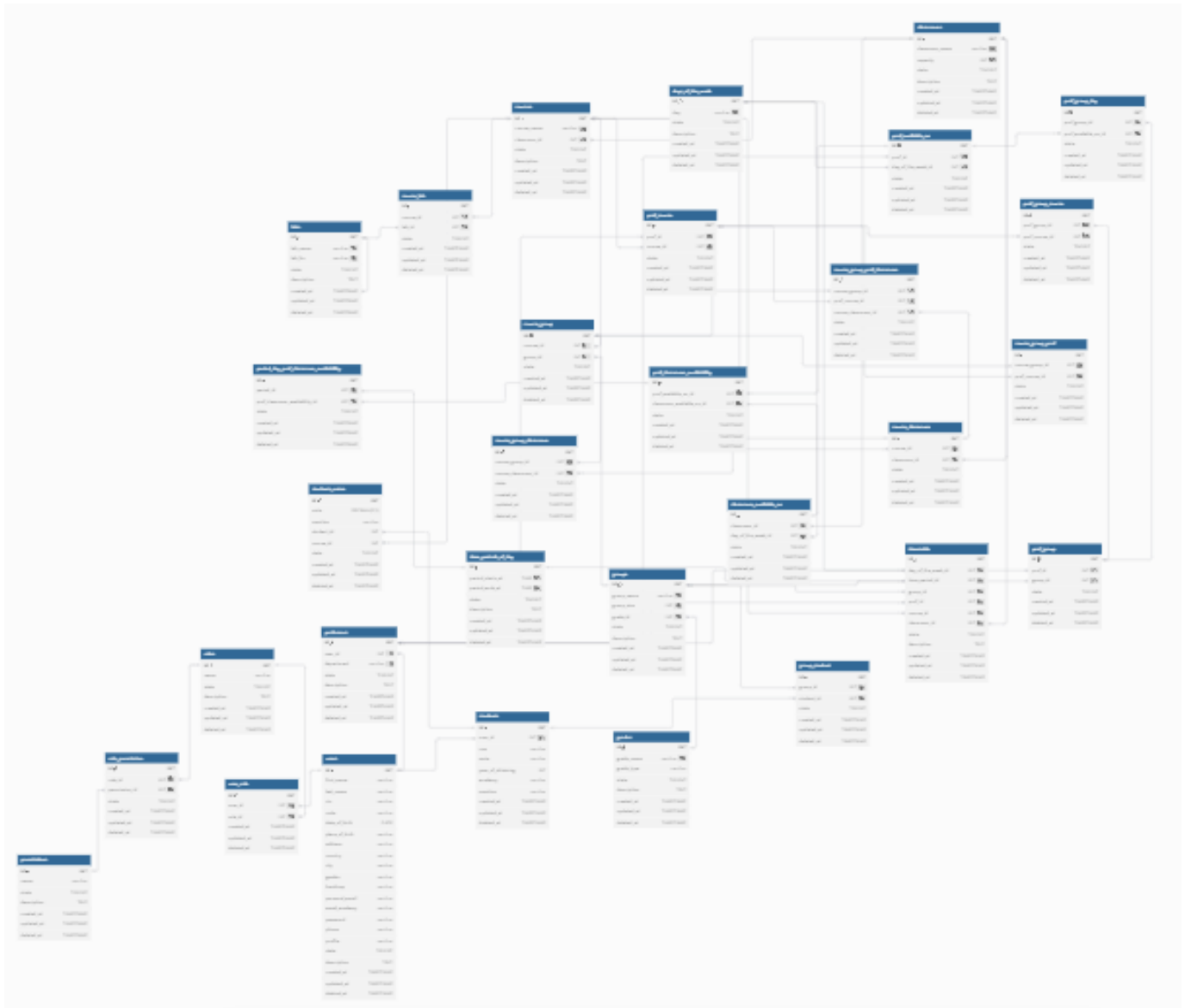


Figure 2 database diagram pro

This database schema is designed to handle the complexities of a university timetabling system, including user management, course scheduling, resource allocation, and student performance tracking. The relationships between tables ensure data integrity and enable efficient querying for generating timetables and managing academic activities.

7. SQL Queries for Data Retrieval and Management

To run the timetabling program effectively, it is essential to retrieve the necessary data from the database after it has been properly populated. Each feature in the model obtains its corresponding values based on predefined attributes outlined in the article. For instance, groups of students are categorized into lower-grade (k1) and higher-grade (k2) groups. The data for these features is queried dynamically from the database where specific conditions, such as `state = 1` (active), are met. These values are stored as arrays or maps in Java, which are then passed to the proposed model to generate solutions or the final timetable. Below are example SQL queries for retrieving data for k1 (lower-grade) and k2 (higher-grade) groups:

- **k1 Query:**

```
SELECT gp.id, gp.group_name, gd.grade_name, gd.grade_type
FROM groups gp
JOIN grades gd ON gp.grade_id = gd.id
WHERE gd.grade_type = "lower-grade" AND gp.state = 1;
```

- **k2 Query:**

```
SELECT gp.id, gp.group_name, gd.grade_name, gd.grade_type
FROM groups gp
JOIN grades gd ON gp.grade_id = gd.id
WHERE gd.grade_type = "higher-grade" AND gp.state = 1;
```

These queries ensure the accurate extraction of relevant data, allowing the timetabling system to operate seamlessly and produce results that align with the university's requirements.

Check rest of queries on:

https://github.com/elidrissilaoukili/timetabling/blob/main/data/selection_queries.sql

8. Data variables

The following table provides a detailed explanation of what each query should select. Each symbol represents a specific set of data or condition that is integral to the proposed timetabling model. This structured approach ensures clarity in data retrieval, allowing the model to operate efficiently and generate optimal timetables:

Symbol	Description
I	Days
J	Periods
K	Groups of students
K1	Lower-grade years
K2	Higher-grade years
L	Teachers
M	Courses
N	Classrooms
K _I	Groups for which teacher I offers some course
L _i	Teachers available on day i
L _k	Teachers teaching at least one course for group k
L _m	Teachers teaching course m
L _{km}	Intersection of L _k and L _m
L _{ki}	Intersection of L _k and L _i
M _k	Courses designed for group k
M _I	Courses taught by teacher I
M _n	Courses fitting in classroom n
M _{kl}	Intersection of M _k and M _I
M _{kn}	Intersection of M _k and M _n
M _{kl_n}	Intersection of M _k , M _I , and M _n
M _{k_{com}}	Elective courses
M _{lab}	Lab courses
N _{mk}	Classrooms fitting group k for course m
I _n	Days classroom n is available
I _I	Days teacher I is available
I _{In}	Intersection of I _I and I _n
J _{iln}	Periods on day i for teacher I and classroom n
JL _{iln}	Time intervals for teacher I and classroom n
FJL _{iln}	Starting periods of intervals
P _m	is the number of repetitions required for a multi-period session of course m
H _m	Length of multi-period sessions
a _k	Total teaching periods for each group
b _m	Total teaching periods required for course m
s _I	Total teaching periods required for teacher I
PRA	A priori assignments

After solving the university timetabling problem and generating an optimal schedule, the output can be inserted into the database for further processing or visualization. The data can then be queried and sent to the frontend for display to end-users, such as professors, students, and administrators. Below is an example query to retrieve and organize the generated timetable for a specific grade and group.

```
PRA = SELECT t.id, day.day, time_period.start_time, time_period.end_time,
        grp.group_name, grd.grade_name, grd.grade_type, prof.full_name,
        course.course_name, course.course_type, classroom.classroom_name, classroom.classroom_type
FROM timetable t
JOIN days_of_the_week day ON day.id = t.day_of_the_week_id
JOIN time_periods_of_day time_period ON time_period.id = t.time_period_id
JOIN groups grp ON grp.id = t.group_id
JOIN grades grd ON grd.id = grp.grade_id
JOIN professors prof ON prof.id = t.prof_id
JOIN courses course ON course.id = t.course_id
JOIN classrooms classroom ON classroom.id = t.classroom_id
WHERE grd.grade_name="semester 1" AND grp.group_name="SMI section A";
```

Why Custom SQL Queries instead of Spring boot JPA ?

For more flexibility, custom SQL queries were used instead of Spring Boot JPA. This approach allowed for fine-tuned control over database operations and optimized query execution according to the specific requirements of the application.

II. Development Tools & project structure

1. Tools:

2. **IntelliJ IDEA 2024.3**: An advanced IDE for efficient Java and Spring Boot development.
<https://www.jetbrains.com/idea/>
3. **MySQL**: A robust relational database management system for storing and managing data.
<https://www.mysql.com/>
4. **Spring Boot**: A Java-based framework for building enterprise-level applications.
<https://start.spring.io/>
5. **IBM CPLEX**: An optimization solver for solving complex scheduling and optimization problems.
<https://www.ibm.com/products/ilog-cplex-optimization-studio>

Github

All resources, including code, documentation, and related materials, are available on our GitHub repository. You can access the project and contribute to its development through the following link:
<https://github.com/elidrissilaoukili/timetabling>

Website

To improve code organization and streamline access to the constraints implemented in the model, I developed a dedicated website. This website categorizes and organizes all constraints systematically, ensuring clarity and ease of reference. Instead of embedding lengthy code snippets directly into the report, I provide a link to the website, which contains the full implementation details of each constraint:
<https://elidrissilaoukili.github.io/timetabling>

This approach not only enhances readability but also allows for efficient updates and accessibility for anyone reviewing or utilizing the model.

2. Project structure:

In the backend, the project is organized under the directory `src/main/java/com/timetabling/system` with the following main components:

- `SystemApplication.java`
- Folders for service, repository, entity, and controller (necessary for CRUD operations)

However, since our goal is to primarily focus on implementing the model, we will mainly work with a terminal-based project for greater flexibility in debugging and testing. This allows us to iterate quickly and ensure the model's functionality without the overhead of full CRUD operations at this stage.

Example how classes will be created using one table:

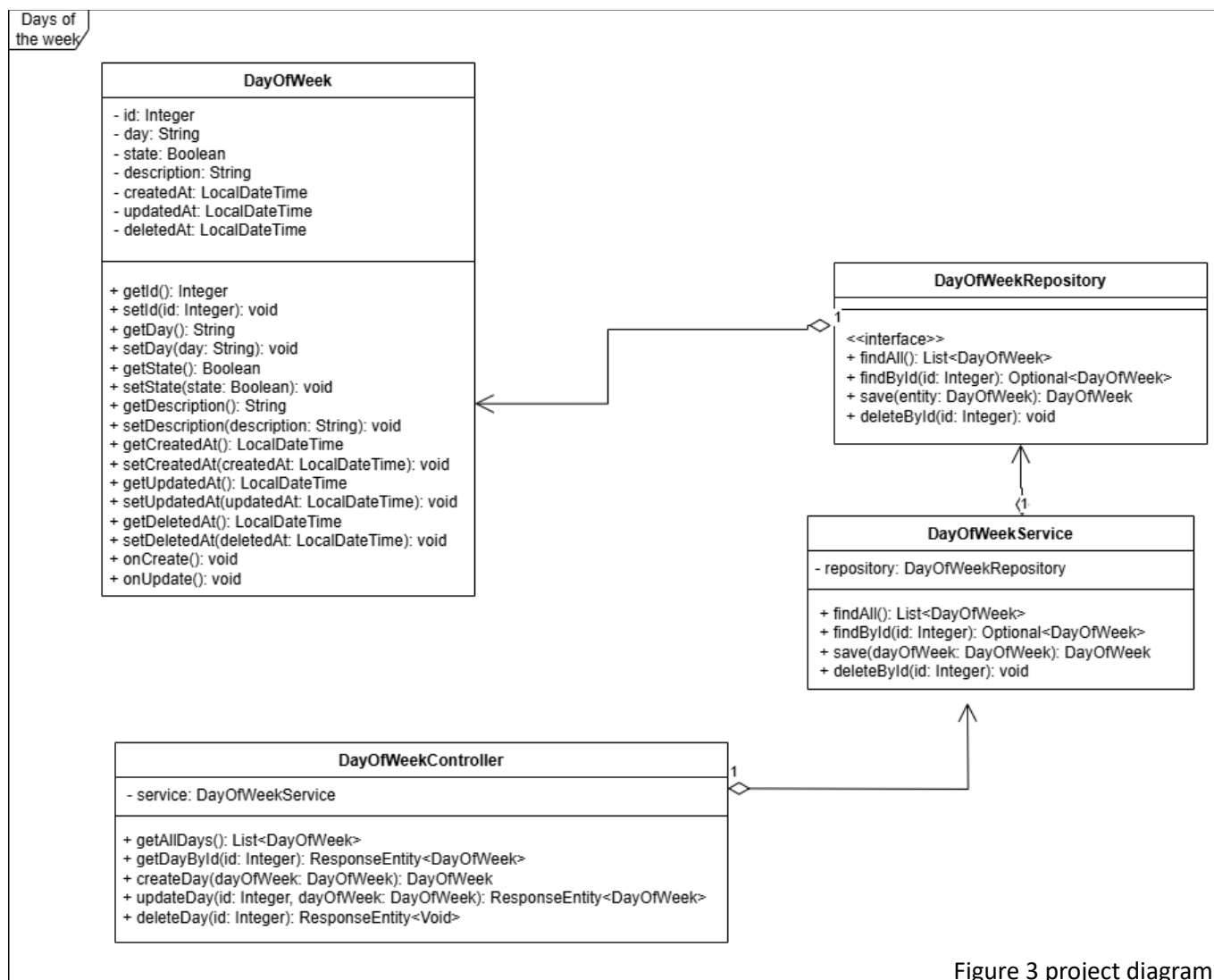


Figure 3 project diagram

To facilitate this, I created a `timetabling` folder containing the following files:

- `Main.java`
- `Timetable.java`
- `TimetablingData.java`

These files handle the core logic of the `timetabling` model and provide a streamlined way to test and debug the application before moving to real data.

III. Testing and Debugging Approach

To ensure the program works as expected before working with real data, I began testing with simple, lightweight data. This approach helps avoid potential memory issues with IBM CPLEX's free version.

The initial tests used basic data in the form of Java arrays, such as:

```
I = new int[]{1}; // days
J = new int[]{1}; // periods
K = new int[]{0, 1}; // groups
K1 = new int[]{0}; // Lower-grade years (Group 0)
K2 = new int[]{1}; // Higher-grade years (Group 1)
L = new int[]{0}; // professors
M = new int[]{0}; // courses
N = new int[]{0}; // classrooms
```

Visit and view from:

<https://elidrissilaoukili.github.io/timetabling/#initializeDatasimple>

<https://elidrissilaoukili.github.io/timetabling/#initializeDataExtended>

Using this minimal data set allowed me to test and debug directly from the terminal, ensuring the scheduling system's logic was functioning correctly. Once I confirmed the expected behavior with this lightweight data, I will proceed with integrating real data for further testing and deployment.

IV. Applying the Proposed Model

To implement the proposed model, I began with a single decision variable, X , and incorporated the first four uniqueness constraints. This approach ensured that the solver operates correctly at each step, allowing me to verify that the basic framework for constraint satisfaction and optimization is functioning as intended. Once this step was validated, I will proceed to more complex scenarios.

1. Timetable Diagram

To visualize the structure of the class `timetable` and how the different components (such as professors, courses, classrooms, and time periods) interact within the scheduling model, refer to the following class diagram:

class Timetable	
- model	: IloCplex
- x	: IloNumVar[][][][][]
- y	: IloNumVar[][][][][]
- data	: TimetablingData
+ Timetable(TimetablingData data)	: <<constructor>>
+ createModel()	: void
- defineDecisionVariables()	: void
- defineAuxiliaryVariables()	: void
- createObjectiveFunction()	: void
- createConstraint1()	: void
- createConstraint2()	: void
- createConstraint3()	: void
- createConstraint4()	: void
- createConstraint5()	: void
- createConstraint6()	: void
- createConstraint7()	: void
- createConstraint8()	: void
- createConstraint9()	: void
- createConstraint10()	: void
- createConstraint11()	: void
- createConstraint12()	: void
- createConstraint13()	: void
- createConstraint14()	: void
- createConstraint15()	: void
- getSolution()	: void
+ solveModel()	: void

class DataFetcher

```
- JdbcTemplate jdbcTemplate;
```

```
// I = days_of_the_week
+ getDayWeek() : short[]
// J = time_periods_of_day
+ getPeriod() : short[]
// K1 = groups from lower-grade
+ getLowerGradeGroups() : List<GroupDTO>
// K2 = groups from higher-grade
+ getHigherGradeGroups() : List<GroupDTO>
// K = groups from both
+ getGroups() : short[]
// L = professors
+ getProfs() : short[]
// M = courses
+ getCourses() : short[]
// N = classrooms
+ getClassrooms() : short[]
// K1 & Lk = group_prof
+ getGroupProf() : List<GroupProfDTO>
// Li & Il = prof_available_on
+ getProfAvOn() : List<ProfessorAvailabilityDTO>
// Lm & Ml = prof_course
+ getProfCourse() : List<ProfessorCourseDTO>
// Mk = course_group
+ getCourseGroups() : List<CourseGroupDTO>
// Lkm & Mkl = prof_group_course
+ getProfGroupCourses() : List<ProfGroupCourseDTO>
// Lki = prof_group_day
+ getProfGroupDays() : List<ProfGroupDayDTO>
// Mn = course_classroom
+ getCourseClassrooms() : List<CourseClassroomDTO>
// Nmk ~ Mkn = course_group_classroom
+ getCourseGroupClassrooms() : List<CourseGroupClassroomDTO>
// Mklm = course_group_prof_classroom
+ getCourseGroupProfClassrooms() : List<CourseGroupProfClassroomDTO>
// Ni = classroom_available_on
+ getClassroomAvOn() : List<ClassroomAvailableOnDTO>
// Lni = prof_classroom_availability
+ getProfClassroomAvOn() : List<ProfClassroomAvailabilityDTO>
// Jlni = period_prof_classroom_availability
+ getPeriodProfClassroomAvOn() : List<PeriodProfClassroomAvailabilityDTO>
// pv
+ getPeriodRepetitionCounts() : short[]
// hv
+ getPeriodLengthCounts() } : short[]
```

class TimetablingData

```

- int[] I          - Map<Integer, int[]> K_l          - int[] M_lab
- int[] J          - Map<Integer, int[]> L_i          - Map<String, int[]> N_mk
- int[] K          - Map<Integer, int[]> L_k          - Map<Integer, int[]> I_n
- int[] K1         - Map<Integer, int[]> L_m          - Map<Integer, int[]> I_l
- int[] K2         - Map<String, int[]> L_km         - Map<String, int[]> I_ln
- int[] L          - Map<String, int[]> L_ki         - Map<String, int[]> J_iln
- int[] M          - Map<Integer, int[]> M_k          - Map<String, int[][]> JL_iln
- int[] N          - Map<Integer, int[]> M_l          - Map<String, int[]> FJL_iln
                  - Map<Integer, int[]> M_n          - Map<Integer, int[]> P_m
                  - Map<String, int[]> M_kl         - Map<Integer, int[]> H_m
                  - Map<String, int[]> M_kn         - Map<Integer, Integer> a_k
                  - Map<String, int[]> M_kln        - Map<Integer, Integer> b_m
                  - int[][] M_k_com              - Map<Integer, Integer> s_l
                                          - int[][] PRA

```

```

+ initializeData() : void          + getK_l()           : Map<Integer, int[]>
+ getI()           : int[]         + getL_i()           : Map<Integer, int[]>
+ getJ()           : int[]         + getL_m()           : Map<Integer, int[]>
+ getK()           : int[]         + getL_km()          : Map<String, int[]>
+ getK1()          : int[]         + getL_ki(int k, int i) : int[]
+ getK2()          : int[]         + getM_k()           : Map<Integer, int[]>
+ getL()           : int[]         + getM_k(int k)       : int[]
+ getM()           : int[]         + getM_l()           : Map<Integer, int[]>
+ getN()           : int[]         + getM_n()           : Map<Integer, int[]>
+ getK_l(int l)    : int[]         + getM_kl()          : Map<String, int[]>
+ getL_k(int k)    : int[]         + getM_kl(int k, int l) : int[]
+ getM_k_com()     : int[][]       + getM_kn()          : Map<String, int[]>
+ getM_lab()       : int[]         + getN_mk(int m, int k) : int[]

```

```

+ getM_kln(int k, int l, int n) : Map<String, int[]>
+ getN_mk()                     : Map<String, int[]>
+ getI_n()                      : Map<Integer, int[]>
+ getI_l()                      : Map<Integer, int[]>
+ getI_ln(int l, int n)         : int[]
+ getJ_iln(int i, int l, int n) : int[]
+ getA_k(int k)                 : int
+ getJL_iln()                   : Map<String, int[][]>
+ getFJL_iln()                  : Map<String, int[]>
+ getP_m()                      : Map<Integer, int[]>
+ getH_m()                      : Map<Integer, int[]>
+ getB_m(int m)                 : int
+ getS_l(int l)                 : int
+ getH_m(int m)                 : int[]
+ getMaxSessionLengths()        : int
+ getPRA()                      : int[][]

```

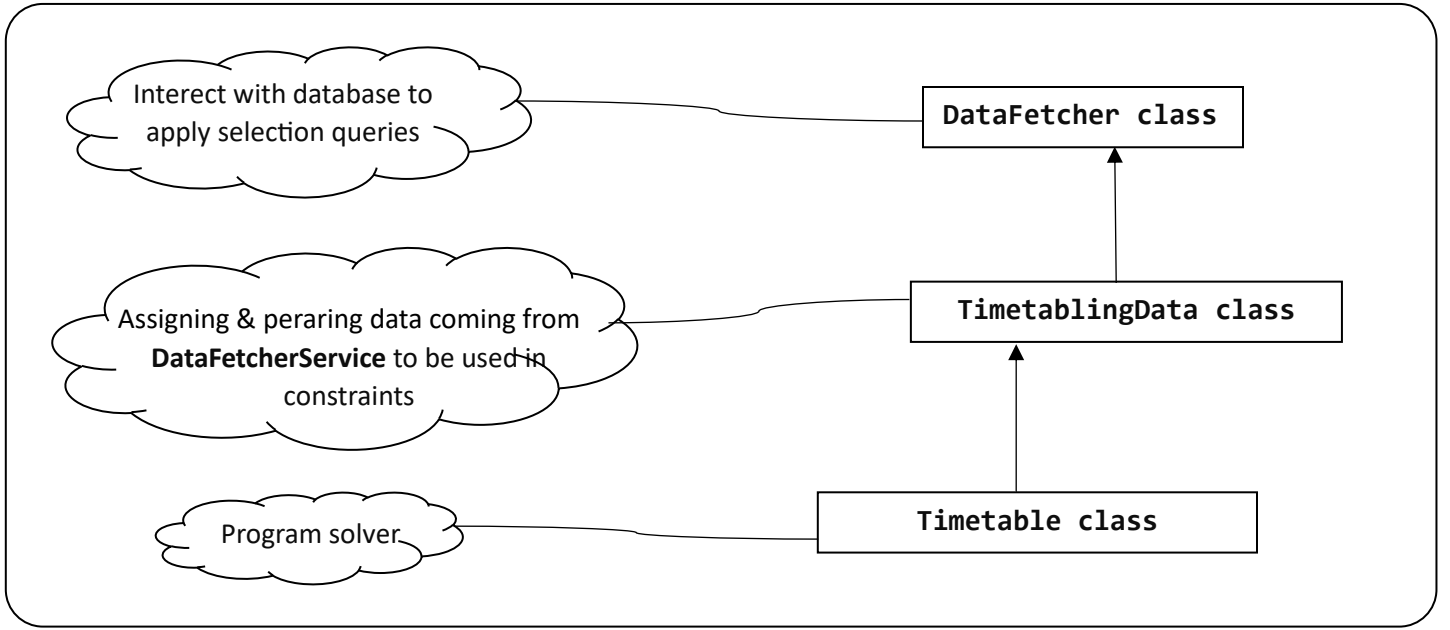


Figure 3 timetable classes interactions

This diagram provides a detailed overview of the main classes involved in the timetabling system and their relationships, which will be helpful as we scale up the model to handle real data and more complex scheduling requirements.

2. Constraints for the IP model

2.1. Uniqueness constraints

$$\forall i \in I, \quad \forall j \in J, \quad \forall l \in L_i, \quad \sum_{k \in K_l} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} x_{i,j,k,l,m,n} \leq 1. \quad (1)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint1>

$$\forall k \in K^1, \quad \forall i \in I, \quad \forall j \in J, \quad \sum_{l \in L_{ki}} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} x_{i,j,k,l,m,n} \leq 1, \quad (2)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint2>

$$\forall k \in K^2, \quad \forall i \in I, \quad \forall j \in J, \quad \sum_{l \in L_{ki}} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} x_{i,j,k,l,m,n} + \sum_{(k_a, l, m) \in M_k^{\text{com}}} \sum_{n \in N_{mka}} x_{i,j,k_a,l,m,n} \leq 1. \quad (3)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint3>

$$\forall n \in N, \quad \forall i \in I_n, \quad \forall j \in J_{in}, \quad \sum_{k \in K} \sum_{l \in L_{ki}} \sum_{m \in M_{kl}} x_{i,j,k,l,m,n} \leq 1. \quad (4)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint4>

After running the program on these constraints, this was the result:

```

Groups for teacher 0: [0, 1]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Courses for group 1 and teacher 0: [0]
Classrooms for course 0 and group 1: [0]
Teachers for group 0 and day 0: []
Teachers for group 1 and day 0: []
Version identifier: 22.1.1.0 | 2022-11-26 | 9160aff4d
Found incumbent of value 0.000000 after 0.00 sec. (0.00 ticks)

Root node processing (before b&c):
  Real time          =    0.00 sec. (0.00 ticks)
Parallel b&c, 4 threads:
  Real time          =    0.00 sec. (0.00 ticks)
  Sync time (average) =    0.00 sec.
  Wait time (average) =    0.00 sec.
  -----
Total (root+branch&cut) =    0.00 sec. (0.00 ticks)
Solution status: Optimal

```

2.2. Completeness constraints

$$\forall k \in K^1, \sum_{l \in L_k} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} \sum_{i \in I_{ln}} \sum_{j \in J_{iln}} x_{i,j,k,l,m,n} = a_k, \quad (5)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint5>

$$\forall k \in K^2, \sum_{l \in L_k} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} \sum_{i \in I_{ln}} \sum_{j \in J_{iln}} x_{i,j,k,l,m,n} + \sum_{(k_a,l,m) \in M_k^{\text{com}}} \sum_{n \in N_{mk_a}} \sum_{i \in I_{ln}} \sum_{j \in J_{iln}} x_{i,j,k,l,m,n} = a_k, \quad (6)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint6>

$$\forall k \in K, \quad \forall l \in L_k, \quad \forall m \in M_{kl}, \quad \sum_{n \in N_{mk}} \sum_{i \in I_{ln}} \sum_{j \in J_{iln}} x_{i,j,k,l,m,n} = b_m, \quad (7)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint7>

$$\forall l \in L, \quad \sum_{k \in K} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} \sum_{i \in I_{ln}} \sum_{j \in J_{iln}} x_{i,j,k,l,m,n} = s_l, \quad (8)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint8>

After running the program on the 8 constraints, this was the result:

```
Groups for teacher 0: [0, 1]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Courses for group 1 and teacher 0: [0]
Classrooms for course 0 and group 1: [0]
Teachers for group 0 and day 0: [0]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Teachers for group 1 and day 0: [0]
Courses for group 1 and teacher 0: [0]
Classrooms for course 0 and group 1: [0]
Teachers for group 0: [0]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Days for teacher 0 and classroom 0: [0]
Periods for day 0, teacher 0, and classroom 0: [0]
Accessing x[0][0][0][0][0][0]
Version identifier: 22.1.1.0 | 2022-11-26 | 9160aff4d
Infeasibility row 'c6': 0 = 1.
Presolve time = 0.00 sec. (0.00 ticks)

Root node processing (before b&c):
  Real time                =    0.00 sec. (0.00 ticks)
Parallel b&c, 4 threads:
  Real time                =    0.00 sec. (0.00 ticks)
  Sync time (average)      =    0.00 sec.
  Wait time (average)      =    0.00 sec.
  -----
Total (root+branch&cut) =    0.00 sec. (0.00 ticks)
No solution found.
```

Why No Solution is Found with completeness constraints ?

Data Limitations:

- Data is very limited:
 - Only 1 day ($I = [0]$).
 - Only 1 period ($J = [0]$).
 - Only 1 teacher ($L = [0]$).
 - Only 1 course ($M = [0]$).
 - Only 1 classroom ($N = [0]$).
- With such limited resources, it is mathematically impossible to satisfy the constraints that require multiple assignments (e.g., assigning a course to a group with $a_k = 1$).

How to Fix This

To resolve the issue and find a feasible solution, you need to **Expand the Data** Add more days (I), periods (J), teachers (L), courses (M), and classrooms (N).

For example:

```
// Days (zero-based indexing)
I = new int[]{0, 1, 2}; // 3 days
// Periods (zero-based indexing)
J = new int[]{0, 1, 2}; // 3 periods
// Teachers (zero-based indexing)
L = new int[]{0, 1}; // 2 teachers
// Courses (zero-based indexing)
M = new int[]{0, 1}; // 2 courses
// Classrooms (zero-based indexing)
N = new int[]{0, 1}; // 2 classrooms
```

Why I did not Expand the Data ?

Due to the limitations of the free version of IBM CPLEX, which has a restriction on the problem size it can handle, I decided not to expand the data at this stage. Expanding the dataset could cause the solver to exceed its size limit, leading to errors or an inability to solve the model. By working with simple and lightweight data, I ensured that the model runs smoothly within these constraints while allowing for thorough testing and debugging before transitioning to more complex scenarios with real data.

I extended data as follow: 

```
public void initializeData() {
    // Days (zero-based indexing)
    I = new int[]{0, 1, 2, 3, 4}; // Day 0

    // Periods (zero-based indexing)
    J = new int[]{0, 1, 2, 3, 5, 6}; // Period 0

    // Groups of students (zero-based indexing)
    K = new int[]{0, 1, 2, 3, 4};
    K1 = new int[]{0}; // Lower-grade years (Group 0)
```

```
K2 = new int[] {1}; // Higher-grade years (Group 1)

// Teachers (zero-based indexing)
L = new int[] {0, 1, 2, 3, 4}; // Teacher 0

// Courses (zero-based indexing)
M = new int[] {0, 1, 2, 3, 4}; // Course 0

// Classrooms (zero-based indexing)
N = new int[] {0, 1, 2, 3, 4};
}
```

But I get the following result: ☹

Checking license ...

C:\Program

Files\IBM\ILOG\CPLEX_Studio_Community2211\cplex\bin\x64_win64\cpxchecklic.
exe: Exiting with code 1

Child process exited with code 1 [0x1]

ilog.cplex.CpxException: CPLEX Error 1016: Community Edition. Problem
size limits exceeded. Purchase at <http://ibm.biz/error1016>.

```
at ilog.cplex.CplexI.CALL(CplexI.java:4815)
at ilog.cplex.CplexI.setStatus(CplexI.java:6300)
at ilog.cplex.CplexI.access$500(CplexI.java:191)
at ilog.cplex.CplexI$SolveHandle.stop(CplexI.java:2998)
at ilog.cplex.CplexI.solve(CplexI.java:3037)
at ilog.cplex.CplexI.solve(CplexI.java:3029)
at ilog.cplex.IloCplex.solve(IloCplex.java:13225)
at timetabling.Timetable.solveModel(Timetable.java:1105)
at timetabling.Main.main(Main.java:14)
```

No license found. [0.14 s]

Process finished with exit code 0

That's why I decided to work with simplified data, even if the solution might not be found with this dataset, due to the feasibility and strict constraints of the model. This approach allows for testing and validation of the system's structure and logic without the complexity of a fully populated dataset.

2.3. Consecutiveness constraints

$$\forall i \in I, \quad \forall k \in K, \quad \forall l \in L_{ki}, \quad \forall m \in M_{kl}, \quad \forall n \in N_{mk},$$
$$\sum_{j \in J_{iln}} x_{i,j,k,l,m,n} - \sum_{h_v \in H_m} \sum_{p_v \in P_m} (y_{i,p_v,k,h_v,m,n} * h_v) = 0. \quad (9)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint9>

$$\forall i \in I, \quad \forall k \in K, \quad \forall l \in L_{ki}, \quad \forall m \in M_{kl}, \quad \forall n \in N_{mk}, \quad \forall j_a \in FJL_{iln}, \quad \forall h_v \in H_m \wedge h_v > 1,$$
$$\forall t \in \{1, \dots, h_v - 1\}, \quad x_{i,j_a,k,l,m,n} - x_{i,j_a+t,k,l,m,n} \leq 0; \quad (10)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint10>

$$\forall i \in I, \quad \forall k \in K, \quad \forall l \in L_{ki}, \quad \forall m \in M_{kl}, \quad \forall n \in N_{mk}, \quad \forall j \in J_{iln}, \quad \forall h_v \in H_m \wedge h_v > 1,$$
$$\forall t \in \{2, \dots, h_v\}, \quad -x_{i,j,k,l,m,n} + x_{i,j+1,k,l,m,n} - x_{i,j+t,k,l,m,n} \leq 0. \quad (11)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint11>

2.4. Repetitiveness constraints

$$\forall k \in K, \quad \forall m \in M_k - M_{lab}, \quad \forall h_v \in H_m, \quad \forall n \in N_{mk}, \quad \sum_{i \in I_{ln}} \sum_{p_v \in P_m} y_{i,p_v,k,h_v,m,n} = b_{m,h_v}, \quad (12)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint12>

$$\forall k \in K, \quad \forall m \in M_k - M_{lab}, \quad \forall l \in L_{km}, \quad \forall h_v \in H_m, \quad \forall n \in N_{mk}, \quad \forall i \in I,$$
$$\sum_{h_v \in H_m} \sum_{p_v \in P_m} y_{i,p_v,k,h_v,m,n} \leq 1. \quad (13)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint13>

$$\forall k \in K, \quad \forall m \in M_{lab}, \quad \forall h_v \in H_m, \quad \sum_{n \in N_{mk}} \sum_{i \in I_n} \sum_{p_v \in P_m} y_{i,p_v,k,h_v,m,n} = p \max_m, \quad (14)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint14>

2.5. Pre-assignment constraints

$$\forall (i, j, k, l, m, n) \in \text{PRA}, \quad x_{i,j,k,l,m,n} = 1, \quad (15)$$

<https://elidrissilaoukili.github.io/timetabling/#createConstraint15>

2.6. Objective function for the IP model

$$\begin{aligned} \text{Minimize} \left\{ \sum_{k \in K} \sum_{l \in L_k} \sum_{m \in M_{kl}} \sum_{n \in N_{mk}} \sum_{i \in I_l} \sum_{j \in J_{iln}} c_{i,j,k,l,m,n} * x_{i,j,k,l,m,n} \right. \\ \left. + \sum_{k \in K} \sum_{i \in I} \sum_{m \in M_k} \sum_{n \in N_{mk}} \sum_{h_v \in H_m} \sum_{p_v \in P_m} a_{i,p_v,k,h_v,m,n} * y_{i,p_v,k,h_v,m,n} \right\}. \end{aligned} \quad (16)$$

<https://elidrissilaoukili.github.io/timetabling/#createObjectiveFunction>

2.7. Running the program using on simple data:

<https://elidrissilaoukili.github.io/timetabling/#initializeDatasimple>

Output

```
Days (I): [0]
Periods (J): [0]
Groups (K): [0, 1]
Teachers (L): [0]
Courses (M): [0]
Classrooms (N): [0]
Groups for teacher 0: [0, 1]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Courses for group 1 and teacher 0: [0]
Classrooms for course 0 and group 1: [0]
Teachers for group 0 and day 0: [0]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Teachers for group 1 and day 0: [0]
Courses for group 1 and teacher 0: [0]
Classrooms for course 0 and group 1: [0]
Teachers for group 0: [0]
Courses for group 0 and teacher 0: [0]
Classrooms for course 0 and group 0: [0]
Days for teacher 0 and classroom 0: [0]
Periods for day 0, teacher 0, and classroom 0: [0]
```

Accessing x[0][0][0][0][0][0]
y dimensions: 1, 3, 2, 2, 1, 1
Accessing y[0][0][0][1][0][0]
Accessing y[0][1][0][1][0][0]
Accessing y[0][2][0][1][0][0]
Invalid indices: 0, 0, 0, 2, 0, 0
Invalid indices: 0, 1, 0, 2, 0, 0
Invalid indices: 0, 2, 0, 2, 0, 0
Accessing y[0][0][1][1][0][0]
Accessing y[0][1][1][1][0][0]
Accessing y[0][2][1][1][0][0]
Invalid indices: 0, 0, 1, 2, 0, 0
Invalid indices: 0, 1, 1, 2, 0, 0
Invalid indices: 0, 2, 1, 2, 0, 0
No lab courses found.
Version identifier: 22.1.1.0 | 2022-11-26 | 9160aff4d
Infeasibility row 'c6': 0 = 1.
Presolve time = 0.00 sec. (0.00 ticks)

Root node processing (before b&c):
Real time = 0.00 sec. (0.01 ticks)
Parallel b&c, 4 threads:
Real time = 0.00 sec. (0.00 ticks)
Sync time (average) = 0.00 sec.
Wait time (average) = 0.00 sec.

Total (root+branch&cut) = 0.00 sec. (0.01 ticks)
No solution found.
Version identifier: 22.1.1.0 | 2022-11-26 | 9160aff4d

Root node processing (before b&c):
Real time = 0.00 sec. (0.00 ticks)
Parallel b&c, 4 threads:
Real time = 0.00 sec. (0.00 ticks)
Sync time (average) = 0.00 sec.
Wait time (average) = 0.00 sec.

Total (root+branch&cut) = 0.00 sec. (0.00 ticks)
No feasible timetable could be generated. Check the constraints and input data.

Process finished with exit code 0

Feasibility and Data Limitations in Timetabling

✚ The feasibility of the timetabling problem is significantly influenced by the scale and complexity of the datasets used for testing. Small-scale datasets with limited resources—such as a small number of professors, classrooms, or time periods—often result in overly constrained problems, leading to infeasible solutions. In contrast, larger datasets that reflect real-world complexity, such as those involving 20 professors, 80 student groups, and 15 classrooms, are more likely to yield feasible timetables. The increased availability of resources in such scenarios allows the model to allocate courses more effectively, avoiding overly constrained solutions and improving the practicality of the results.

Due to limitations in IBM CPLEX access during testing, the model was primarily evaluated using smaller datasets. However, with full access to IBM CPLEX, the model can be tested on real-world datasets to better assess its performance under realistic conditions. To facilitate this, I have prepared a dataset based on my university's timetabling system, which is accessible via the provided link.

<https://github.com/elidrissilaoukili/timetabling/blob/main/data/data-feeder.sql>

or use the following database already feeded with data:

https://github.com/elidrissilaoukili/timetabling/blob/main/data/db_timetable-full.sql

For more visual checks:

<https://elidrissilaoukili.github.io/timetabling/webfiles/database.html>

3. Proposed Real-World Datasets for Testing

To evaluate the model on real-world data, I propose the following datasets:

3.1. Lower-Grade Dataset:

✚ This dataset is based on the timetabling system of the Department of Mathematics and Computer Science at my university for lower-grade students. It includes the following:

s1		s3	
SMI section A, 1	prof de Math 1	SMI section A, 2	prof de info 1
SMI section A, 1	prof de Math 2	SMI section A, 2	prof de info 2
SMI section A, 1	prof de Math 3	SMI section A, 2	prof de info 3
SMI section A, 1	prof de Physic 1	SMI section A, 2	prof de info 4
SMI section A, 1	prof de Physic 2	SMI section A, 2	prof de Math 1
SMI section A, 1	prof de info 1	SMI section A, 2	prof de Physic 1
		s5	
SMI section B, 1	prof de Math 1	SMI section A, 3	prof de info 1
SMI section B, 1	prof de Math 2	SMI section A, 3	prof de info 2
SMI section B, 1	prof de Math 3	SMI section A, 3	prof de info 3
SMI section B, 1	prof de Physic 1	SMI section A, 3	prof de info 4
SMI section B, 1	prof de Physic 2	SMI section A, 3	prof de info 5
SMI section B, 1	prof de info 1	SMI section A, 3	prof de info 6

I = days of the week

id	days
1	Monday
2	Tuesday
3	Wednesday
4	Thursday
5	Friday
6	Saturday

J = time periods of a day

id	periods morning	id	periods afternoon
1	08:00 - 09:00	5	02:00 - 03:00
2	09:00 - 10:00	6	03:00 - 04:00
3	10:00 - 11:00	7	04:00 - 05:00
4	11:00 - 12:00	8	05:00 - 06:02

K = groups

id	groups
1	SMI section A, 1
2	SMI section B, 1
3	SMI section A, 2
4	SMI section A, 3

L = professors

id	info profs	id	Math profs	id	physics profs
1	prof de info 1	7	prof de Math 1	10	prof de Physic 1
2	prof de info 2	8	prof de Math 2	11	prof de Physic 2
3	prof de info 3	9	prof de Math 3		
4	prof de info 4				
5	prof de info 5				
6	prof de info 6				

M = courses

id	courses in s1	id	courses in s3	id	courses in s5
1	SMI S1 module 1 Math	13	SMI S3 module 1 info	29	SMI S5 module 1 info
2	SMI S1 module 1 Math TD	14	SMI S3 module 1 info TD	30	SMI S5 module 1 info TD
3	SMI S1 module 2 Math	15	SMI S3 module 1 info TP	31	SMI S5 module 1 info TP
4	SMI S1 module 2 Math TD	16	SMI S3 module 2 info	32	SMI S5 module 2 info
5	SMI S1 module 3 Math	17	SMI S3 module 2 info TD	33	SMI S5 module 2 info TD
6	SMI S1 module 3 Math TD	18	SMI S3 module 2 info TP	34	SMI S5 module 2 info TP
7	SMI S1 module 4 Physic	19	SMI S3 module 3 info	35	SMI S5 module 3 info
8	SMI S1 module 4 Physic TD	20	SMI S3 module 3 info TD	36	SMI S5 module 3 info TD
9	SMI S1 module 5 Physic	21	SMI S3 module 3 info TP	37	SMI S5 module 3 info TP
10	SMI S1 module 5 Physic TD	22	SMI S3 module 4 info	38	SMI S5 module 4 info
11	SMI S1 module 6 info	23	SMI S3 module 4 info TD	39	SMI S5 module 4 info TD
12	SMI S1 module 6 info TD	24	SMI S3 module 4 info TP	40	SMI S5 module 4 info TP
		25	SMI S3 module 5 Math	41	SMI S5 module 5 info
		26	SMI S3 module 5 Math TD	42	SMI S5 module 5 info TD
		27	SMI S3 module 6 Physic	43	SMI S5 module 5 info TP
		28	SMI S3 module 6 Physic TD	44	SMI S5 module 6 info
				45	SMI S5 module 6 info TD
				46	SMI S5 module 6 info TP

N = classrooms

id	for lectures	id	for recitations	id	labs
1	Amphi H1, Amphi	6	class L1, TD	13	A101, TP
2	Amphi H2, Amphi	7	class L2, TD	14	A102, TP
3	Amphi M, Amphi	8	class L4, TD	15	i106, TP
4	Amphi E, Amphi	9	class L5, TD	16	i107, TP
5	Amphi F, Amphi	10	class L6, TD	17	i111, TP
		11	class L7, TD	18	i114, TP
		12	class L8, TD		

3.2. Higher-Grade Dataset:

👉 This dataset reflects the timetabling requirements for higher-grade students in the same department. Specifically, it is based on the Master's program in Machine Learning and Artificial Intelligence (MLAIM), which I am currently pursuing. The dataset includes:

Use the days, periods, classrooms from previous tables:

L	M	Pm	Hm	N	K
professors	courses	periods repitition	session repetition	classroom_id	group
prof 1	Infographie	4 periods	1	18	master mlaim
prof 2	Traitement d'images	4 periods	1	18	
prof 3	Probabilité et processus stochastique	4 periods	1	18	
prof 4	Probabilité et processus markovien	4 periods	1	18	
prof 5	RO	4 periods	1	18	
prof 6	Gouvernance de données	4 periods	1	18	
prof 7	Méta-Heuristiques				
prof 8	Soft Skills	2 periods	1	7	
prof 9	Langues Etrangères 1 (FRANÇAIS)	2 periods	1	7	
prof 10	Langues Etrangères 1 (ANGLAIS)				

These datasets are designed to capture the complexity and resource allocation challenges of real-world timetabling scenarios, providing a robust foundation for testing and validation.

V. Running timetabling application with spring boot

After successfully completing and testing the program in the terminal—debugging and ensuring it works as expected—we will enhance the user experience by presenting the output through a simple yet effective frontend. The User Interface will display the generated timetable neatly and interactively on the endpoint: <http://localhost:8080/api/timetable/solve>. The interface will be designed to make the data visually appealing, with clear formatting, colors for better readability, and potentially export options (e.g., PDF or Excel).

```
package com.timetabling.system.controller;

import com.timetabling.system.timetabling.OutputCapturer;
import com.timetabling.system.timetabling.TimetablingData;
import com.timetabling.system.timetabling.Timetable;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/timetable")
public class TimetableController {

    @GetMapping("/solve")
    public ResponseEntity<String> solveTimetable() {
        // Step 1: Initialize the data
        TimetablingData data = new TimetablingData();
        data.initializeData();

        // Step 2: Create the timetable model
        Timetable timetable = new Timetable(data);

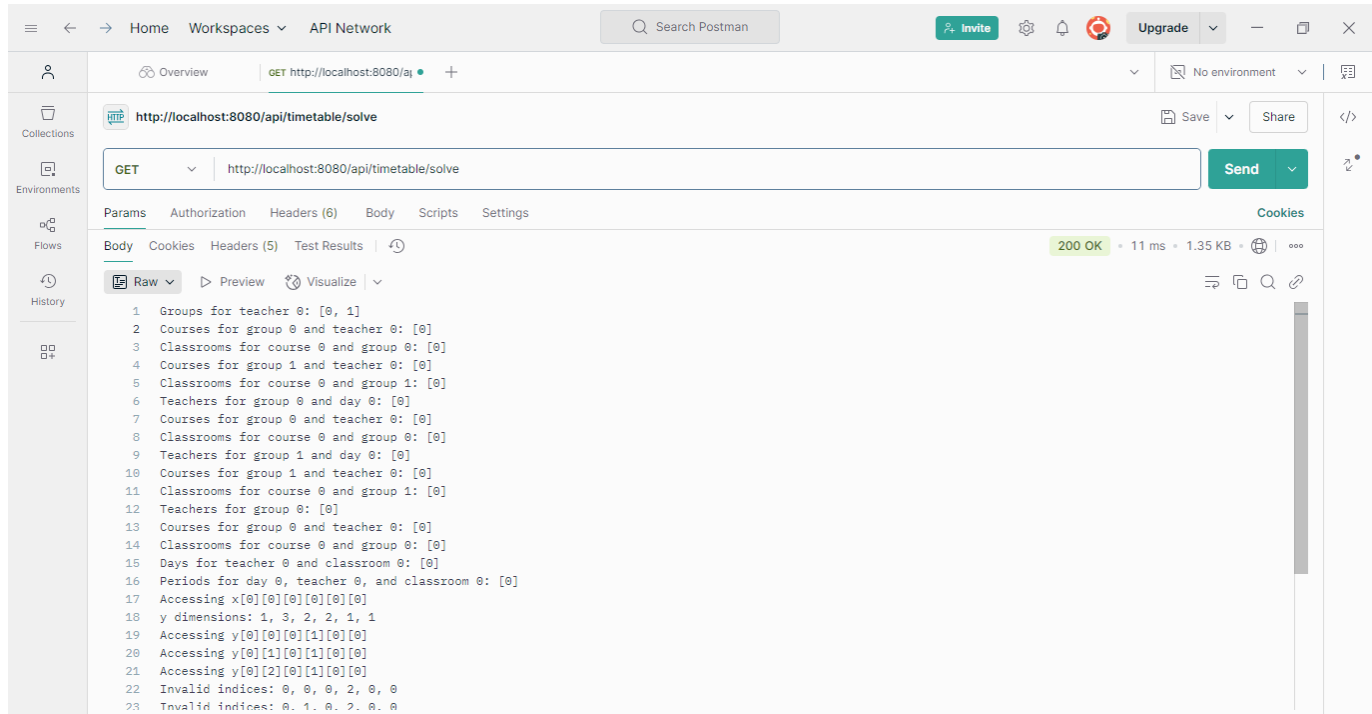
        // Step 3: Capture System.out output
        OutputCapturer capturer = new OutputCapturer();
        capturer.startCapture();

        // Step 4: Create and solve the model
        timetable.createModel();
        timetable.solveModel();

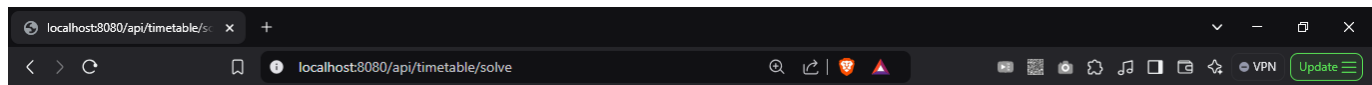
        // Step 5: Stop capturing and get the output
        String output = capturer.stopCapture();

        // Step 6: Return the captured output as the HTTP response
        return ResponseEntity.ok(output);
    }
}
```

Using postman:



Using browser:



Groups for teacher 0: [0, 1] Courses for group 0 and teacher 0: [0] Classrooms for course 0 and group 0: [0] Courses for group 1 and teacher 0: [0] Classrooms for course 0 and group 1: [0] Teachers for group 0 and day 0: [0] Courses for group 0 and teacher 0: [0] Classrooms for course 0 and group 0: [0] Teachers for group 1 and day 0: [0] Courses for group 1 and teacher 0: [0] Classrooms for course 0 and group 1: [0] Teachers for group 0: [0] Courses for group 0 and teacher 0: [0] Classrooms for course 0 and group 0: [0] Days for teacher 0 and classroom 0: [0] Periods for day 0, teacher 0, and classroom 0: [0] Accessing x[0][0][0][0][0][0] y dimensions: 1, 3, 2, 2, 1, 1 Accessing y[0][0][0][1][0][0] Accessing y[0][1][0][1][0][0] Accessing y[0][2][0][1][0][0] Invalid indices: 0, 0, 0, 2, 0, 0 Invalid indices: 0, 1, 0, 2, 0, 0 Invalid indices: 0, 2, 0, 2, 0, 0 Accessing y[0][0][1][1][0][0] Accessing y[0][1][1][1][0][0] Accessing y[0][2][1][1][0][0] Invalid indices: 0, 0, 1, 2, 0, 0 Invalid indices: 0, 1, 1, 2, 0, 0 Invalid indices: 0, 2, 1, 2, 0, 0 No lab courses found. No solution found. No feasible timetable could be generated. Check the constraints and input data.



Conclusion

This project has been a challenging yet rewarding experience that deepened my understanding of solving real-world problems using technology. The university timetabling problem, while seemingly straightforward, proved to be complex due to the numerous constraints that needed to be considered, such as class schedules, teacher availability, and room capacity.

I approached this challenge methodically. Initially, I decided to simplify the process by bypassing the use of a database and directly sending inputs from the React frontend to the Spring Boot backend for processing. This decision allowed me to focus on the core logic of timetable generation without being bogged down by CRUD operations.

The complexity of the project came from identifying and applying the constraints correctly. Each constraint represented a unique challenge, and balancing them required careful planning and optimization. To address this, I avoided CRUD operations and frontend data input entirely, instead opting to instantiate the required data directly within the program using a dedicated **TimetablingData** class. This approach allowed me to focus entirely on building the logic and ensuring the constraints were properly formulated before implementing a solver to generate feasible schedules. Leveraging tools like CPLEX made it possible to handle these constraints effectively and produce an optimized timetable.

This project taught me the importance of breaking down a problem into manageable parts. By focusing on the essentials first, such as constraint formulation and logic implementation, I was able to build a strong foundation before improving the system with a simple yet intuitive user interface. The output, displayed at *localhost:8080/api/timetable/solve*, provided a clear and user-friendly view of the generated timetable, showcasing the practical value of the solution.

Overall, I found this project both intellectually stimulating and deeply fulfilling. It reinforced my problem-solving skills, my ability to tackle complexity, and my confidence in applying advanced tools and frameworks like Spring Boot and CPLEX. It was **not just** about building a solution, **it was about learning how to navigate challenges, think critically, and deliver results.**

Resources:

 GitHub Repository: <https://github.com/elidrissilaoukili/timetabling>

 Project Website : <https://elidrissilaoukili.github.io/timetabling>